TAF

1. Ajouter un nouvel sorcier ‘Wizard’
Créez une méthode qui enregistre un sorcier à l’aide de `NewWizard`, puis lui attribue un identifiant.

2. Filtrer par maison ‘House’
Créez la méthode `findByHouse(house: House)` en utilisant le référentiel générique ou un service.

3. Ajouter un événement de sort ‘Spell’
Utilisez `Extract<AcademyEvent, { type: 'spell-cast' }>` pour définir un type pour les événements de lancement de sorts.

4. Valider la puissance d'un sort ‘Spell Power’
Créez un décorateur qui rejette un sort lorsque sa puissance est inférieure à zéro.

5. Ajouter une méthode mise en cache
Créez un décorateur de méthode qui met en cache les résultats précédents à l'aide d'une Map.

6. Rendre la validation de type plus stricte ‘Type Guard’
Vérifiez que house correspond à l'une des quatre valeurs House autorisées au lieu d'accepter n'importe quelle chaîne de caractères.

TAF

Exercice 1 — Ajouter un nouvel sorcier

Objectif : enregistrer un assistant sans que l'appelant ait à fournir d'identifiant.

Vous disposez déjà de ce utility type dans le fichier types.ts :

```
export type NewWizard = Omit<Wizard, "id">;
```

Omit<Wizard, "id"> supprime la propriété « id » du type Wizard. Cela signifie que l'appelant fournit toutes les autres propriétés obligatoires, et que c'est le référentiel ou le service qui attribue l'identifiant.
Puis, faire un test dans main.ts

TAF

Exercice 2 — Filtrer les sorciers par maison

Objectif : Récupérer tous les sorciers appartenant à une maison spécifique de Poudlard.

Étape 1 : Ajouter la méthode à **WizardRepository** Dans **src/repository.ts**, modifiez la classe **WizardRepository** :

```
findByHouse(house: Wizard["house"]): Wizard[] {
  return this.getAll().filter(
    wizard => wizard.house === house
  );
}
```

Ici, Wizard["house"] est un type à accès indexé. Il extrait le type de la propriété « house » de Wizard, qui correspond à votre union « House ».Cela signifie que la méthode accepte "Gryffindor", "Slytherin", "Ravenclaw“ ou "Hufflepuff", mais pas une chaîne de caractères arbitraire.

Puis, faire un test

TAF

Exercice 3 — Utilisation d'Extract pour gérer les événements de lancement de sorts
Objectif : créer un gestionnaire qui n'accepte que les événements de lancement de sorts, même si l'académie prend en charge plusieurs types d'événements.

Votre type `AcademyEvent` actuel est une union discriminée :

```
export type AcademyEvent =
  { type: "spell-cast"; wizardId: number; spellName: string }
  | { type: "level-up"; wizardId: number; newLevel: number }
  | { type: "wizard-expelled"; wizardId: number };
```

Étape 1 : Extraire le type d'événement « spell-casting »Dans src/types.ts, ajoutez :

```
export type SpellCastEvent = Extract<AcademyEvent,{ type: "spell-cast" }>;
```

TAF

Exercice 3 — Utilisation d'Extract pour gérer les événements de lancement de sorts
Objectif : créer un gestionnaire qui n'accepte que les événements de lancement de sorts, même si l'académie prend en charge plusieurs types d'événements.

Étape 2 : Créer un eventHandler
Ajoutez cette fonction dans src/main.ts :

```ts
import type { SpellCastEvent } from "./types";

function handleSpellCast(event: SpellCastEvent): void {
  console.log(
    `Wizard ${event.wizardId} cast ${event.spellName}`
  );
}

handleSpellCast({
  type: "spell-cast",
  wizardId: 1,
  spellName: "Expelliarmus"
});
```

TAF

Exercice 4 — Vérification de la puissance d'un sort à l'aide d'un décorateur de méthode

Objectif : Refuser un sort si sa puissance est négative.

Nous allons conserver l'API de décorateur existante et appliquer le décorateur à la méthode add() de SpellRepository.

Étape 1 : Implémenter **ValidateSpellPower,** Ajoutez ceci dans src/decorators.ts :

```
export function ValidateSpellPower(
  target: object,
  propertyKey: string | symbol,
  descriptor: PropertyDescriptor): void {
  const originalMethod = descriptor.value as (
    ...args: unknown[]
  ) => unknown;

  descriptor.value = function (this: unknown,...args:unknown[]): unknown {
    const candidate: unknown = args[0];

    if (typeof candidate === "object" && candidate !== null
        && "power" in candidate && typeof candidate.power === "number" &&
      candidate.power < 0 ) {
      throw new Error("Spell power cannot be negative");
    }

    return originalMethod.apply(this, args);
  };
}
```

TAF

Exercice 4 — Vérification de la puissance d'un sort à l'aide d'un décorateur de méthode

Objectif : Refuser un sort si sa puissance est négative.

Étape 2 : Appliquer ces modifications au repository, Dans src/repository.ts :

```
import { ValidateSpellPower } from "./decorators";

export class SpellRepository extends Repository<Spell> {
  @ValidateSpellPower
  override add(spell: Spell): void {
    super.add(spell);
  }
}
```

TAF

Exercice 4 — Vérification de la puissance d'un sort à l'aide d'un décorateur de méthode

Objectif : Refuser un sort si sa puissance est négative.

Étape 3 : Tester les sorts valides et non valides

```
spells.add({
  id: 102,
  name: "Protego",
  category: "defense",
  power: 50
});

spells.add({
  id: 103,
  name: "Invalid Spell",
  category: "attack",
  power: -10
});
```

TAF

Exercice 5 — Créer un décorateur de méthode avec mise en cache

Objectif : éviter de recalculer le même résultat lorsqu'une méthode reçoit plusieurs fois les mêmes arguments.

Dans cet exercice, nous allons créer un décorateur pour les méthodes synchrones.

Il mettra en cache les résultats séparément pour chaque instance d'objet.

Étape 1 : Mise en place du cache, Ajoutez ceci dans le fichier src/decorators.ts :

```ts
export function Cache(
  target: object,
  propertyKey: string | symbol,
  descriptor: PropertyDescriptor
): void {
  const originalMethod = descriptor.value as (...args: unknown[]) => unknown;

  const instanceCaches = new WeakMap<object,Map<string, unknown>>();

  descriptor.value = function (this: object,...args: unknown[]): unknown {
    let cache = instanceCaches.get(this);
    if (!cache) {
      cache = new Map<string, unknown>();
      instanceCaches.set(this, cache);
    }
    const key = JSON.stringify(args);
    if (cache.has(key)) {
      console.log(`[CACHE HIT] ${String(propertyKey)}`);
      return cache.get(key);
    }
    console.log(`[CACHE MISS] ${String(propertyKey)}`);
    const result = originalMethod.apply(this, args);
    cache.set(key, result);
    return result;
  };
}
```

TAF

Exercice 5 — Créer un décorateur de méthode avec mise en cache

Objectif : éviter de recalculer le même résultat lorsqu'une méthode reçoit plusieurs fois les mêmes arguments.

Étape 2 : Appliquer le décorateur

Créez un service dans src/main.ts :

```ts
import { Cache } from "./decorators";

class SpellCalculator {
  @Cache
  calculatePower(basePower: number, multiplier: number): number {
    console.log("Calculating...");

    return basePower * multiplier;
  }
}

const calculator = new SpellCalculator();

console.log(calculator.calculatePower(10, 3));
console.log(calculator.calculatePower(10, 3));
console.log(calculator.calculatePower(5, 2));
```

TAF

Exercice 6 — Renforcer le Type Guard
Objectif : s'assurer que la maison d'un sorcier fait partie de l'une des quatre maisons valides de Poudlard, plutôt que d'accepter n'importe quelle chaîne de caractères.

Dans votre code source, le Type Guard `isWizard()` vérifiait que « house » était une chaîne de caractères. Cela aurait conduit à accepter à tort « Durmstrang » comme maison valide.

Étape 1 : Créer un type guard dédié à isHouse()Dans src/main.ts :

Le type de retour « value is House » est un prédicat de type. Il indique à TypeScript que, lorsque la fonction renvoie « true », la valeur « value » peut être considérée sans risque comme une « House ».

```ts
import type { House, Wizard } from "./types";

const validHouses: readonly House[] = [
  "Gryffindor",
  "Slytherin",
  "Ravenclaw",
  "Hufflepuff"
];

function isHouse(value: unknown): value is House {
  return (
    typeof value === "string" &&
    validHouses.some(house => house === value)
  );
}
```

TAF

Exercice 6 — Renforcer le Type Guard

Objectif : s'assurer que la maison d'un sorcier fait partie de l'une des quatre maisons valides de Poudlard, plutôt que d'accepter n'importe quelle chaîne de caractères.

Étape 2 : Mise à jour de la fonction isWizard()

Remplacez votre implémentation précédente par :

```
function isWizard(value: unknown): value is Wizard {
  if (typeof value !== "object" || value === null) {
    return false;
  }

  const candidate = value as Record<string, unknown>;

  return (
    typeof candidate.id === "number" &&
    typeof candidate.name === "string" &&
    isHouse(candidate.house) &&
    typeof candidate.level === "number" &&
    typeof candidate.active === "boolean"
  );
}
```

TAF

Exercice 6 — Renforcer le Type Guard
Objectif : s'assurer que la maison d'un sorcier fait partie de l'une des quatre maisons valides de Poudlard, plutôt que d'accepter n'importe quelle chaîne de caractères.

Étape 3 : Tester les cas valides et non valides

```
const validData: unknown = {
  id: 1,
  name: "Harry Potter",
  house: "Gryffindor",
  level: 5,
  active: true
};

const invalidData: unknown = {
  id: 2,
  name: "Viktor",
  house: "Durmstrang",
  level: 5,
  active: true
};

console.log(isWizard(validData));   // true
console.log(isWizard(invalidData)); // false
```

Ce que vous allez mettre en pratique



Conditional Types & Infer
Dérivez des types en fonction des entrées, extrayez les résultats de fonctions et filtrez les types d'union.



Utility Types
Utilisez Pick, Omit, Partial, Record, Parameters, ReturnType, Extract et Exclude.



Index Signatures
Stockez des statistiques de sorts et des valeurs de configuration flexibles.



Type Guards
Distinguez les enregistrements de sorciers des enregistrements de sorts et gérez en toute sécurité les données inconnues.



Method Decorators
Enregistrez les appels de méthodes, mesurez le temps d'exécution et validez l'accès aux sorts restreints.



Generics
Créez un référentiel réutilisable capable de stocker des assistants, des sorts et d'autres entités.